

Sitzer –

Requirements Specification and Test Plan

Spis treści

1. Introduction	3
2. Business Objectives.....	3
3. Functional Requirements.....	4
4. Technologies and Architecture	7
5. Test Plan	8
6. Test Environment.....	8
7. Test Scenarios	9
8. Defects and Error Analysis in Application Modules.....	10

1. Introduction

- Project Objective

The application was created to practically apply knowledge in the field of mobile application development and software testing.

- System Scope

A fitness & health application with an offline-first approach.

- Brief Application Description

Sitzer is an application designed for people who want to improve their posture and reduce the negative effects of prolonged sitting.

- Target User Group

The application's target users are people who lead a sedentary lifestyle, mainly office workers, computer gamers, and people whose daily duties and work require spending long hours in a sitting position.

2. Business Objectives

The application provides a solution to problems related to maintaining consistency in performing exercises and starting to incorporate postural exercises into everyday life. Using specially designed workout plans, potential users can strengthen or stretch muscles that have previously caused discomfort. The accessible form of a mobile application makes a device such as a smartphone a tool for introducing healthy daily habits and reducing the effects of prolonged sitting.

3. Functional Requirements

- **User Authentication (Login / Register / Logout)**

ID	Requirement	Priority
FR-Auth-01	The system allows a new user to register by providing a name, email address, password, and password confirmation.	High
FR-Auth-02	The system validates that the name field is not empty before registration.	High
FR-Auth-03	The system validates the email address format (must contain @ and a domain).	High
FR-Auth-04	The system requires the password to be at least 6 characters long.	High
FR-Auth-05	The system verifies that the password matches the password confirmation field.	High
FR-Auth-06	The system displays an error message when attempting to register with an existing email address.	High
FR-Auth-07	The system allows users to log in using their email address and password.	High
FR-Auth-08	The system displays an appropriate error message when incorrect login credentials are provided.	High
FR-Auth-09	The login button is enabled only when both fields (email and password) are filled in.	Medium
FR-Auth-10	The system redirects the logged-in user to the main screen (HomeScreen).	High
FR-Auth-11	The system allows the user to log out from the profile screen.	Medium
FR-Auth-12	User data is stored locally in the Room database.	High

- **Home Screen (HomeScreen)**

ID	Requirements	Priority
FR-HOME-01	The home screen displays the user's current daily workout plan.	High
FR-HOME-02	The screen presents workout progress in the form of statistics cards.	High
FR-HOME-03	The screen displays a personalized greeting with the logged-in user's name.	Medium
FR-HOME-04	The screen allows navigation to the workout plan selection screen (PlanSelectScreen).	High
FR-HOME-05	The home screen is accessible only to logged-in users.	High
FR-HOME-06	The screen allows navigation to the workout hub (WorkoutHubScreen).	High

- **Workout Plans (PlanSelectScreen)**

ID	Requirements	Priority
FR-PLAN-01	The system provides three workout plan levels: Beginner, Intermediate, and Advanced.	High
FR-PLAN-02	Each plan includes a description, difficulty level, and estimated duration.	Medium
FR-PLAN-03	The user can select and activate a workout plan.	High

FR-PLAN-04	The currently selected plan is visually highlighted in the list.	Medium
FR-PLAN-05	The system stores the selected workout plan in DataStore (persistence across sessions).	High
FR-PLAN-06	The screen provides filters for searching and narrowing down the list of workout plans.	Medium
FR-PLAN-07	The selected filters are preserved after closing the filter panel.	Medium
FR-PLAN-08	The plans include information about the muscle groups they target.	Niski

- **Workout Hub (WorkoutHubScreen)**

ID	Requirements	Priority
FR-WORK-01	The screen displays a list of exercises from the currently selected workout plan.	High
FR-WORK-02	Each exercise displays its name, description, number of repetitions, or duration.	High
FR-WORK-03	The exercises include a demonstration video or animation showing the correct execution.	High
FR-WORK-04	The system provides a workout timer with an audio/visual signal upon completion.	Medium
FR-WORK-05	The user can mark an exercise as completed.	High
FR-WORK-06	The system tracks the user's daily exercise progress.	Medium
FR-WORK-07	The exercise video is displayed correctly after the device screen is rotated.	High
FR-WORK-08	The system supports 30+ different exercises in the database.	Medium

- **User Profile (ProfileScreen)**

ID	Requirements	Priority
FR-PROF-01	The screen displays the logged-in user's information: name and email address.	High
FR-PROF-02	The user can change their name through the edit panel (Bottom Sheet).	Medium
FR-PROF-03	The user can change their email address through the edit panel.	Medium
FR-PROF-04	The user can change their password; the change is stored in the Room database.	High
FR-PROF-05	The system validates data entered into the profile editing fields (empty fields, email format).	High
FR-PROF-06	The user can add or change their profile picture (avatar).	Low
FR-PROF-07	The system displays an appropriate placeholder when the image file is corrupted or unavailable.	Medium
FR-PROF-08	The profile editing panel (Bottom Sheet) preserves its data after screen rotation.	Medium
FR-PROF-09	The user's name is displayed correctly after switching accounts.	Medium
FR-PROF-10	The profile editing fields are validated against SQL Injection.	High

- **Ustawienia**

ID	Requirements	Priority
FR-SETT-01	The system provides three theme modes: Light, Dark, and Follow System.	Wysoki
FR-SETT-02	The selected theme is applied immediately after selection without restarting the application.	Średni
FR-SETT-03	The selected theme is stored in DataStore and restored after restarting the application.	Wysoki
FR-SETT-04	The system provides a choice of interface language: Polish and English (i18n).	Wysoki
FR-SETT-05	The language change is applied throughout the application without restarting it.	Wysoki
FR-SETT-06	The selected language is stored in DataStore and restored after restarting the application.	Wysoki
FR-SETT-07	Changing the system language does not override the language selected by the user in the settings.	Średni
FR-SETT-08	The settings panel (Bottom Sheet) preserves its state after screen rotation.	Średni
FR-SETT-09	On devices with API ≥ 33 , changing the language uses <code>AppCompatDelegate.setApplicationLocales()</code> .	Wysoki
FR-SETT-10	The system allows enabling and disabling workout reminder notifications.	Średni
FR-SETT-11	The user can configure the notification schedule (time and frequency).	Średni
FR-SETT-12	The Help panel (Help Bottom Sheet) is displayed correctly and preserves its state after screen rotation.	Niski

- **Notifications and Navigation (Other)**

ID	Wymaganie	Priorytet
FR-NOTI-01	The system creates Android notification channels (NotificationChannel) when the application starts.	High
FR-NOTI-02	Notifications are scheduled using WorkManager (background task scheduling).	High
FR-NOTI-03	The notifications panel (Notifications Bottom Sheet) preserves its settings after screen rotation.	Medium
FR-NOTI-04	The splash screen (SplashScreen) displays a logo animation and a progress bar for approximately 3 seconds.	Low
FR-NOTI-05	After the SplashScreen finishes, the application automatically redirects the user to the LoginScreen.	High
FR-NOTI-06	Navigation between screens uses Jetpack Navigation Compose with a Single Activity architecture.	High
FR-NOTI-07	Navigating back through the navigation stack preserves the correct screen state.	Medium
FR-NOTI-08	The bottom navigation is visible on all main screens after logging in.	Medium

4. Technologies and Architecture

- **Architecture**

MVVM – Separation of layers and responsibilities within the application.

Single Activity + Jetpack Navigation – A single activity containing the main component, with screen transitions managed in an organized manner.

Unidirectional Data Flow – One-way data flow based on StateFlow.

- **Technologies used**

Kotlin – Main programming language.

Jetpack Compose – Library for declarative user interface development.

Hilt – Dependency Injection (DI) to provide better modularity in ViewModel files and object passing between layers.

Room + SQLite – Local database

DataStore – Storage of configuration data (key-value).

WorkManager – Background tasks (e.g., notification system).

Jetpack Navigation Compose – Navigation between screens and application flow management.

StateFlow / LiveData – Application state management.

Coroutines – Asynchronous operations within the application, performing background operations to avoid blocking the main thread.

5. Test Plan

The main objective of the testing process for this project is to identify all issues related to the core functionalities of the MVP version, gain practical experience in manual testing, learn the fundamentals of UI and integration test automation, and validate application behavior. The entire testing process has been transferred to Jira for the practical application of a sprint-based workflow and task management methodology. Strategia testowania

- **Manual testing**
- **Integration testing (JUnit, Mockito)**
- **Integration testing (Room DB)**
- **UI/Espresso testing (automation fundamentals)**
- **Security testing (SQL Injection)**
- **i18n testing (Polish/English language support implemented in the application)**
- **Recomposition and data persistence testing**

6. Test Environment

- Android Studio (emulator + device)
- Pixel 5 — API 33 (emulator)
- Medium Phone — API 36 (emulator)
- Xiaomi Mi Mix 2s (physical device)
- Jira (project management)
- Git / GitHub (version control)
- ADB (debugging)

7. Test Scenarios

- **Test Organization**

To reflect a proper working environment in Jira, the project uses a structure based on epics, stories, and dedicated sprints.

Epiki

- **SITZERB4RE-9 – User Authentication Testing**

LoginScreen Manual Testing, RegisterScreen Manual Testing, LoginScreen Unit Testing, RegisterScreen Unit Testing.

Coverage: Login, registration, field validation, SQL Injection, recomposition, and navigation.

- **SITZERB4RE-7 – Workout Module Testing**

PlanSelectScreen Manual Testing, PlanSelectScreen Unit Testing, WorkoutHub Manual Testing, WorkoutHub Unit Testing.

Coverage: Workout plan filters, plan selection, video playback, timer, component recomposition, and data persistence.

- **SITZERB4RE-8 – Profile & Settings Testing**

ProfileScreen Manual Testing, ProfileScreen Unit Testing.

Coverage: Profile editing, password/email changes, avatar, theme, language, notifications, and SQL Injection.

- **SITZERB4RE-64– Home Testing**

HomeScreen Manual Testing, HomeScreen Unit Testing

Coverage: Onboarding, categories, workout plan view, navigation, recomposition state, and data persistence.

- **SITZERB4RE-10 – Cross-module Testing**

Theme Manual / Unit Testing, Notification Unit Testing, Navigation Manual/Unit Testing, Onboarding Manual/Unit Testing

Coverage: Cross-module testing between screens, data persistence, and settings persistence across application components.

- **SITZERB4RE-2 – Environment Setup**

Coverage: Configuration of emulators, test data, a stable testing branch, and the CI environment.

- **Test Cases**

All test cases are available in the **jira_test-cases.xlsx** file located in the project's **docs** folder.

8. Defects and Error Analysis in Application Modules

After the application testing process was completed, **19 defects** were identified, which will now be presented in the form of tables. In this case, the discovered defects perfectly illustrate the **Pareto Principle** in software testing, as approximately **80% of the defects were found in 20% of the application's components**, primarily within the authentication system and user profile functionality.

As examples, I will present several reported defects from the Jira system, along with the actions I would take to resolve them.:

- **Moduł autoryzacji logowanie/rejestracja**

Title	[BUG-LOG01] Incorrect translation of the error message on the login and registration screen
Description	An i18n issue. During an unsuccessful login or registration attempt, despite the application and system language being set to English, the displayed error message is hardcoded in Polish.
Test steps / Test data	1. Open the application. 2. Wait until the splash screen finishes loading. 3. Enter the test email address into the Email field. 4. Enter the test password into the Password field. 5. Press the "Login" button. Email: jozefskowron91@gmail.com Password: jozefskowron123
Expected result	The displayed error message should match the currently selected application or system language.
Actual result	The displayed error message is always shown in Polish, regardless of the selected language settings.

Title	[BUG-REG01] User can register an account using an incorrectly formatted email address
Description	Email validation issue. If the entered value resembles the syntax of an email address, its validity is not properly verified.

Test steps / Test data	1. Open the application. 2. Wait until the splash screen finishes loading. 3. Tap the highlighted "Sign In" text. 4. Enter the test username into the Name field. 5. Enter the test email into the Email field. 6. Enter the test password into the Password field. 7. Re-enter the password into the Confirm Password field. 8. Press the "Sign In" button. E-mail: testmail!!^%@ppppp[Username: testuser123 Password: test123pass
Expected result	The registration screen should display an appropriate validation message, and the account should not be created.
Actual result	The user successfully registered a new account using an invalid email format, and no validation message was displayed.

Title	[BUG-REG02] User can register an account using an extremely long password (>200 characters)
Description	Password validation issue. Although password complexity rules exist, allowing passwords longer than 200 characters may cause unexpected issues and should be restricted.
Test steps / Test data	1. Open the application. 2. Wait until the splash screen finishes loading. 3. Tap the highlighted "Sign In" text. 4. Enter the test username into the Name field. 5. Enter the test email into the Email field. 6. Enter the test password into the Password field. 7. Re-enter the password into the Confirm Password field. 8. Press the "Sign In" button. E-mail: jozefskowron91@gmail.com Username: skowron91 Password:jooooozteteaetaetttttttoofozefskoozzzzzzzzzzzzzztttttetefffffffffffffff ffteooooowron91passssssssssss s%%%%%%%%ssssword12345678910101010!!!!
Expected result	The registration screen should display an appropriate validation message, and the account should not be created.
Actual result	The user successfully registered an account using an excessively long password, and no validation message was displayed.

For the reported defect **BUG-LOG01**, which was classified as **low priority**, the solution is straightforward. After examining the source code, it can be observed that the error message is hardcoded in Polish within the login/registration method.

```
else {
    errorMessage = "Zły email lub hasło"
    loginSuccess = false
}
```

For **BUG-REG01** and **BUG-REG02**, the solution was also quickly identified. By reviewing the user registration validation logic, it became apparent that the only password validation rule checks whether the password contains fewer than five characters, while the email format validation is not implemented correctly.

```
fun registerUser() { 11 Usages
    //Podstawowa walidacja dla rejestracji
    if (firstName.isEmpty() || email.isEmpty() || password.isEmpty() || confirmPassword.isEmpty()) {
        errorMessage = "Uzupełnij wszystkie pola!"
        return
    }

    if (password != confirmPassword) {
        errorMessage = "Hasła nie są identyczne!"
        return
    }

    if (password.length < 5) {
        errorMessage = "Hasło musi mieć co najmniej 5 znaków!"
        return
    }
}
```

• WorkoutHub Module

Title	[BUG-WOR01] Changing the screen orientation while displaying an exercise card causes incorrect rendering
Description	Changing the device orientation during an active workout causes the exercise screen to render incorrectly. The exercise presentation card and navigation panel become hidden, and the layout of the UI elements is disrupted.
Test steps / Test data	1. Log into the application using the provided test credentials. 2. Select "Plans" from the bottom navigation bar. 3. Select the "Core" category. 4. Open the "Core Crusher" workout plan. 5. Press the "Start Workout" button in the bottom sheet. 6. Press the "Next" button twice. 7. Press the "Start" button. 8. Rotate the device into landscape orientation. E-mail: jozefskowron91@gmail.com Password: jozefskowron123

Expected result	The " Crunches " exercise video should continue playing correctly, while the exercise title and description remain visible and properly positioned.
Actual result	After rotating the device, the exercise card containing the video, title, and description becomes collapsed and hidden.

Testing screen behavior and field values during recomposition proved valuable, as the **WorkoutHubScreen** did not correctly handle landscape orientation. The root cause was the layout implementation, which relied on `weight(1f)` values. While these values worked correctly in portrait mode, they caused layout inconsistencies in landscape orientation. The proposed solution is to make the main column scrollable by applying the scrollable modifier.

```
{ paddingValues ->
  val scrollState = rememberScrollState()
  Column(
    modifier = Modifier
      .fillMaxSize()
      .verticalScroll(scrollState)
      .padding(paddingValues)
      .padding( all = 16.dp),
    horizontalAlignment = Alignment.CenterHorizontally
  ) {
```

• Profile and Settings Module

Title	[BUG-PRO02] Attempting to set a corrupted image as the user avatar results in an incorrectly displayed placeholder
Description	A corrupted image generated using the " Online Corrupt File Generator " website causes the default profile placeholder to be displayed incorrectly after being selected as the profile picture.
Test steps / Test data	1. While logged in, open the "Profile" tab from the bottom navigation bar. 2. Tap the Edit (pencil) icon next to the profile picture. 3. Tap "Select Picture". 4. Choose the corrupted image file corrupted_img.jpg from the gallery. 5. Press "Select". 6. Press "Save" in the bottom sheet. E-mail: jozefskowron91@gmail.com Password: jozefskowron123  corrupted_img(1).jpg

Expected result	The application should display an appropriate error message and restore the default profile image.
Actual result	No error message is displayed, and the profile picture area becomes empty instead of showing the default placeholder.

Title	[BUG-PRO09] Saved application language settings are overridden after changing the system language
Description	After selecting a preferred application language, changing the device's system language causes the application language to switch to the system language after the application is restarted.
Test steps / Test data	1. Open the "Profile" tab. 2. Open Settings. 3. Select "Polish" as the application language. 4. Press "Save". 5. Close the application. 6. Change the device system language to English. 7. Launch the application again. 8. After the splash screen, navigate to the login screen. E-mail: jozefskowron91@gmail.com Password: jozefskowron123
Expected result	The application language should remain set to Polish , as previously selected in the application settings, regardless of the system language.
Actual result	The application automatically switches its language to match the system language.

The reported defect **BUG-PRO02** was resolved by properly configuring the parameters passed to the method responsible for processing the new profile picture. The **Coil** library provides a straightforward mechanism for defining both the image displayed during loading and the fallback image shown whenever an error occurs while loading the selected image.

```
val avatarPainter = avatarUri?.let {
    rememberAsyncImagePainter(
        model = it,
        placeholder = painterResource(id = R.drawable.pfpp),
        error = painterResource(id = R.drawable.pfpp)
    )
} ?: painterResource(id = R.drawable.pfpp)
```